CS 486/686
From Prompting to Fine-Tuning and LoRA

Yuntian Deng

Lecture 22

How to adapt and augment a language model

Learning goals

- Choose the right adaptation strategy.
- Explain **RAG** and **tool use**.
- Explain **SFT**, preference tuning, and **LoRA**.
- Evaluate risks and failure modes.

From L21 to L22

A fixed model plus a prompt can do a lot.

```
CS486 helper: Is Assignment 2 due before or after
Chat 9?
```

What if the answer needs fresh course facts,
a calculation, or a durable behavior change?

Running task: CS486 course helper

We want an assistant that answers course questions accurately and cites course material.

accurate

uses current course facts

helpful

uses the right format

grounded

shows its sources

The adaptation ladder

1. better prompt
2. retrieved context (RAG)
3. tools / actions
4. supervised fine-tuning
5. LoRA / adapters

Use the cheapest lever that solves the problem.

Diagnose before choosing the method

Need style?
prompt or SFT

Need fresh/private facts?
RAG

Need calculation/action?
tools

Need durable behavior?
SFT or LoRA

Prompting changes context, not weights

general

Explain gradient descent.

targeted

Explain gradient
descent to a 10-year-old.

A prompt can steer behavior, but cannot add missing knowledge reliably.

System prompts set behavior defaults

```
You are a concise CS486 TA. Answer using only
course material when possible. Cite the source for
deadlines.
```

Useful, but still limited by what is in the context.

A prompt template is a small program

role	CS486 teaching assistant
task	answer the student question
constraints	cite sources, say if unsure
format	short answer + evidence

When prompting is the wrong fix

missing facts

not in context

tool needs

must calculate or act

brittleness

phrasing changes behavior

RAG motivation

Question: **When is Assignment 2 due?**

A base model does not know this course webpage.

Retrieve evidence first, then answer.

RAG in one sentence

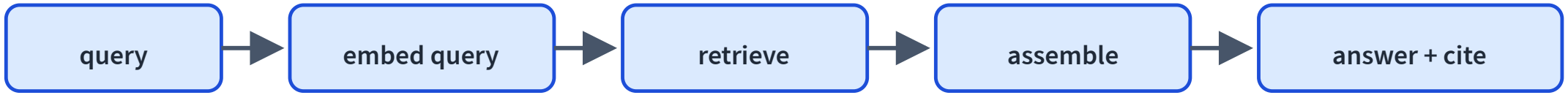
Retrieve relevant documents at inference time, insert them into the prompt, then generate an answer grounded in those documents.

RAG changes the context, not the model weights.

RAG: offline indexing

RAG quality starts before the user asks a question.

RAG: online query path



RAG on real course facts LIVE

Embed the question, retrieve the nearest course chunks, then ground the answer.

When is Assignment 2 due?

retrieve

Could not load the course index.

Embeddings reappear

Documents and queries become vectors.

Nearest vectors are retrieved.

This is L18's embedding geometry doing useful work.

Production RAG is usually hybrid

vector

semantic matches

keyword

exact terms

reranker

precision boost

RAG can still fail

wrong chunk

bad retrieval

missing chunk

low recall

ignored evidence

bad generation

RAG reduces hallucination; it does not eliminate it.

Evaluate retrieval and generation separately

context precision

were retrieved chunks relevant?

context recall

did we retrieve enough?

faithfulness

is answer supported?

answer relevance

did it answer?

Some tasks need actions

calculate

weighted grade

execute

run code

lookup

calendar/API

If the model needs to act, give it a tool.

Tool use loop

The model proposes an action; the environment returns evidence.

Tool use, step by step LIVE

The model calls a calculator instead of guessing the arithmetic.

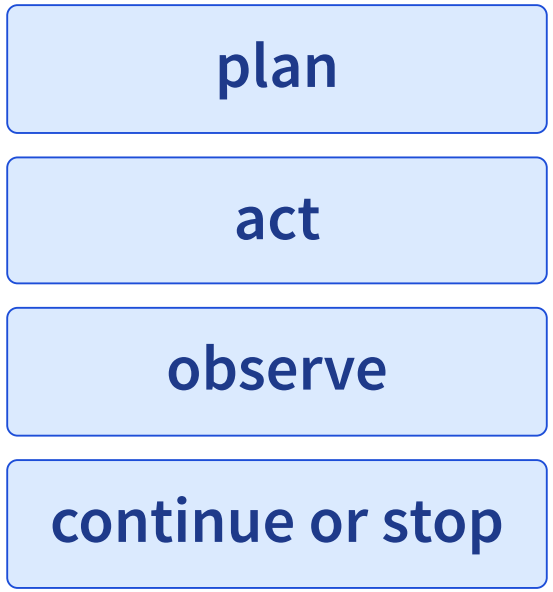
The model proposes an action; the environment (a calculator) returns real evidence; the model uses it.

step reset

user

What is my weighted grade? Assignments 85 (30%), chats 92 (20%), final 78 (50%).

Agents are loops, not magic



Tool use needs guardrails

wrong call

bad action

prompt injection

tool output

attacks prompt

destructive action

needs approval

When should we fine-tune?

Prompting, RAG, and tools change inputs.

Fine-tuning changes weights.

Use it when behavior must change consistently across many examples.

Supervised fine-tuning

Train on curated instruction-response examples.

instruction	ideal response
Answer with citation.	Assignment 2 is due Thu Jul 9 [schedule].
Be concise.	Chat 9 is released Thu Jul 9.

Same next-token loss, targeted dataset.

SFT code sketch

```
for prompt, response in dataset:
    text = chat_template(prompt, response)
    loss = next_token_loss(model, text)
    loss.backward()
    optimizer.step()
```

Pretraining's loss, but on curated behavior examples.

Fine-tuning risks

overfit

small dataset

forget

lose general behavior

artifacts

learn quirks

Use held-out eval and high-quality data.

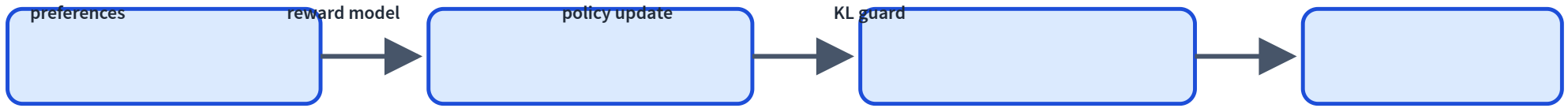
Preferences, not just demonstrations

Sometimes two answers are plausible, but one is better.

```
(prompt, preferred answer, rejected answer)
```

Preference data teaches ranking.

RLHF at high level



Powerful, but engineering-heavy.

DPO simplifies preference tuning

Direct Preference Optimization skips the reward-model/PPO loop.

Increase probability of the preferred answer relative to the rejected answer.

Common in open-model post-training because it is simpler and stable.

Full fine-tuning can be expensive

A large model contains huge weight matrices.

Updating all of them is costly.

Can we learn a small update instead?

LoRA learns a low-rank update

$$W' = W_0 + \Delta W$$

$$\Delta W = BA, \quad r \ll d$$

Freeze W_0; train the small matrices A and B.

LoRA forward pass

$$h = W_0 x + \frac{\alpha}{r} B A x$$

W_0: frozen base weight

A, B: trainable adapter matrices

r: adapter rank

α: scaling factor

LoRA: a low-rank update LIVE

Drag the rank: a small BA update, a fraction of the parameters.

Freeze the big weight W_0; train only a low-rank update $\Delta W = BA$. Higher rank = more expressive but more parameters.

rank r **4**

trainable: **192** vs full **576** (rank r=4, d=24)

only **33%** of the weights are trained

Why LoRA is practical

small

few trainable parameters

cheap

less memory/storage

modular

swap or merge adapters

LoRA code sketch

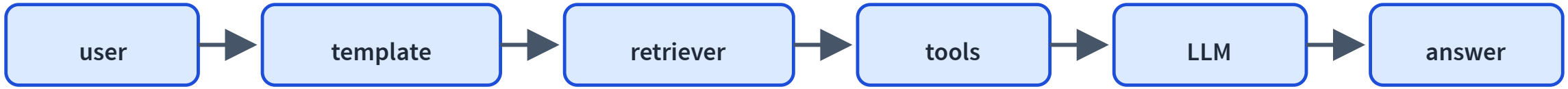
```
config = LoraConfig(
    r=8,
    target_modules=["q_proj", "v_proj"],
)
model = get_peft_model(base_model, config)
train(model, dataset)
```

Looks like fine-tuning; only adapter weights update.

Which lever for the course helper?

failure	best first lever
wrong tone / format	prompt template
missing course fact	RAG
grade calculation	tool
repeated rubric behavior	SFT / LoRA

A practical LLM system



Fine-tuning or LoRA can live inside the LLM block.

System safety checklist

verify context

do not trust
retrieved text blindly

cite sources

show uncertainty

approve actions

human for
irreversible steps

What comes next?

Text-only LMs are one kind of model.

How do models answer questions about images?

L23: Dissecting a Vision-Language Model.

Recap

✓ Prompting changes input, not weights.

✓ RAG adds external knowledge at inference time.

✓ Tools let the model act or compute.

✓ SFT changes weights with demonstrations.

✓ Preference tuning learns from comparisons.

✓ LoRA adapts weights efficiently with low-rank updates.